

Exercise 5: Numerical Proof of Unconditional Stability

$$C^{n+1} = C^n - k \Delta t C^{n+1}$$

$$C^{n+1} + k \Delta t C^{n+1} = C^n$$

$$C^{n+1} (1 + k \Delta t) = C^n$$

$$C^{n+1} = \frac{C^n}{(1 + k \Delta t)}$$

So, $\frac{1}{1 + k \Delta t} \leq 1$ (to maintain stability)

decay rate (k) will be > 0

and the time (Δt) will be > 0

Using an "amplification argument"

is a mathematical way to answer one simple question:

"If I let this computer model run, is it going to explode?"

$$G = \frac{1}{1 + k \Delta t}$$

$|G| < 1$; the simulation remains strictly bounded

$$C^{n+1} = G \cdot C^n; \text{ where } G = \frac{1}{1 + k \Delta t}$$

where $|G| > 1$; the model quickly spirals to ∞ and crashes

where $|G| \leq 1$; the values of decay remain perfectly bounded
preserve physical realism without blowing up for
every Δt value no matter how large.

so, $0 < \frac{1}{1 + k \Delta t} < 1$

Figure 2 graphically proves this.

```

# Parameters
t0 = 0.
tn = 20.
k = 1.0 # decay rate (day^-1), fixed
C0 = 0.5 # mg chl/m3
alpha = 1.0 # alpha=1 => pure backward Euler

# Timesteps to test - large dt values that would destroy forward Euler
dt_list = [0.001, 0.1, 0.5, 1.0, 2.0, 2.2, 5.0, 10.0]

# Analytical solution on fine grid
t_exact = np.linspace(t0, tn, 300)
C_exact = C0 * np.exp(-k * t_exact)

# --- Plot ---
fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(t_exact, C_exact, 'k-', linewidth=2, label='Analytical solution', zorder=5)

colors = ['steelblue', 'seagreen', 'darkorange', 'orchid', 'tomato']

for dt, color in zip(dt_list, colors):
    Ntot = int(np.floor((tn - t0) / dt)) + 1
    C = np.zeros(Ntot)
    t = np.zeros(Ntot)
    C[0] = C0
    t[0] = t0

    # Amplification factor G for backward Euler (alpha=1):
    # C[n+1] = G * C[n], where G = 1 / (1 + alpha*k*dt)
    G = 1.0 / (1.0 + alpha * k * dt)

    for n in range(Ntot - 1):
        C[n+1] = G * C[n] # backward Euler step
        t[n+1] = t[n] + dt

    ax.plot(t, C, 'o--', color=color, linewidth=1.4,
           label=f'Backward Euler Δt={dt} d (G={G:.3f})')

ax.set_xlabel('Time (days)', fontsize=12)
ax.set_ylabel('Concentration (mg chl m-3)', fontsize=12)
ax.set_title('Unconditional Stability of the Backward Euler Scheme', fontsize=13)
ax.legend(fontsize=9, loc='upper right')
ax.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

```

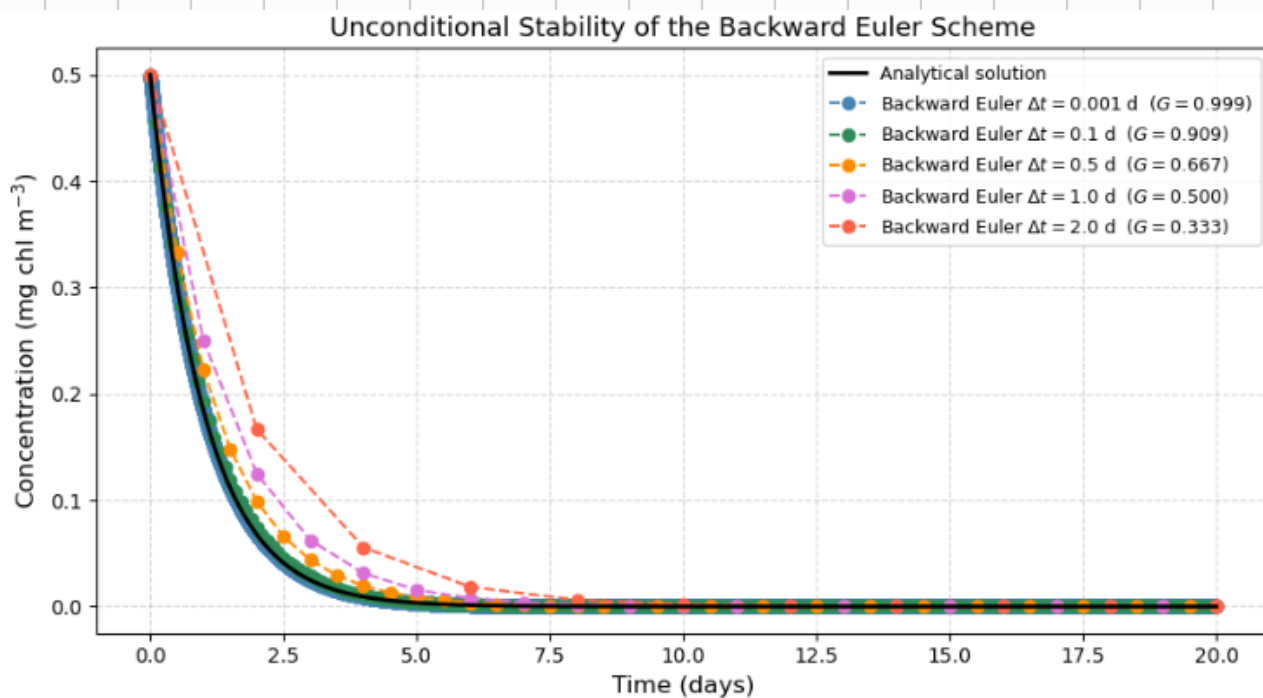


Figure 2: Unconditional Stability of the Backward Euler Scheme